

OS作业5-郭威威-2414308

Linux内核添加新系统调用实验报告

一、实验基本信息

项目	内容
实验名称	Linux内核添加新系统调用（sys_schello）及用户态测试
实验环境	- 操作系统：Ubuntu（或其他基于Debian的Linux发行版） - 内核版本：linux-6.17.1（自定义配置内核） - 编译器：gcc（通过build-essential 套件安装） - 架构支持：x86_64（默认） / arm64（可选）
实验工具	gcc、make、dmesg、内核源码编辑工具（Vim/VS Code）
核心目标	1. 在Linux 6.17.1内核中新增系统调用 sys_schello 2. 实现系统调用的内核态逻辑（打印内核版本、学号） 3. 编写用户态程序调用新系统调用并验证结果

二、实验原理

系统调用是**用户态程序与内核态交互的核心接口**，Linux内核通过固定的“系统调用表”管理所有合法系统调用，新增系统调用需遵循以下核心逻辑：

- 1. **声明系统调用函数**：在内核头文件中声明函数原型，确保内核编译时能识别该函数；
- 2. **实现系统调用逻辑**：在内核源码中编写系统调用的具体功能（如打印日志）；
- 3. **注册系统调用号**：为新调用分配唯一的“系统调用号”（避免冲突），并更新系统调用表和调用号定义；
- 4. **重新编译内核**：将新代码整合到内核中，生成新内核镜像并安装；
- 5. **用户态测试**：通过 `syscall()` 函数（按调用号）触发新系统调用，通过内核日志（`dmesg`）验证结果。

三、实验步骤（详细操作）

前置准备：安装依赖工具

首先安装编译内核和程序所需的基础工具：

代码块

```
1 sudo apt-get install build-essential # 包含gcc、make等编译工具
```

同时确保已获取**linux-6.17.1内核源码**（Step0要求的“第三节课自定义配置内核”），并进入内核源码根目录（后续所有内核修改操作均在此目录执行）。

Step1：声明系统调用函数（头文件修改）

目的：在内核头文件中声明 `sys_schello` 函数，确保内核编译时能找到函数原型。

操作文件：`include/linux/syscalls.h`

具体步骤：

- 6. 用Vim打开文件：`vim include/linux/syscalls.h`；
- 7. 定位到**第1220行附近**（`#endif /* CONFIG_ARCH_HAS_SYSCALL_WRAPPER */`之前）；
- 8. 添加以下代码（声明系统调用函数）：

代码块

```
1 asmlinkage long sys_schello(void); // asmlinkage表示从栈读取参数，符合系统调用规范
```

9. 保存退出：按 `Esc`，输入 `:wq` 并回车。

Step2：实现系统调用逻辑（内核源码修改）

目的：编写 `sys_schello` 的具体功能，打印内核版本号和学号（内核态日志）。

操作文件： `kernel/sys.c`

具体步骤：

10. 用Vim打开文件： `vim kernel/sys.c`；

11. 定位到**第1008行附近**（ `SYSCALL_DEFINE0(gettid)` 函数之后）；

12. 添加以下代码（实现系统调用，注意替换 `123456` 为自己的真实学号）：

代码块

```
1 // 无参数系统调用（DEFINE0表示0个参数），返回0
2 SYSCALL_DEFINE0(schello)
3 {
4     // 打印内核日志：KERN_INFO为日志级别，包含内核版本和学号
5     printk(KERN_INFO "Hello new system call schello! Linux Kernel Version:
6     6.17.1\n");
7     printk(KERN_INFO "Hello new system call schello! Student ID:
8     123456\n"); // 替换为实际学号
9     return 0; // 系统调用成功返回0
10 }
```

13. 保存退出： `:wq`。

Step3：注册系统调用表（架构相关配置）

目的：将 `sys_schello` 添加到系统调用表，让内核通过“调用号”找到该函数。

不同架构的系统调用表路径不同，需根据实际架构选择：

情况1：x86_64架构（主流PC）

操作文件： `arch/x86/entry/syscalls/syscall_64.tbl`

14. 打开文件： `vim arch/x86/entry/syscalls/syscall_64.tbl`；

15. 查看文件**最后一行的调用号**（假设最后一行为 `469 common xxx sys_xxx`），新调用号为“最后一行数字+1”（示例为 `470`）；

16. 在文件末尾添加以下代码：

代码块

```
1 470 common schello sys_schello // 格式: 调用号 类型 调用名 函数名
```

情况2: arm64架构 (嵌入式设备)

操作文件: `arch/arm64/tools/syscall_64.tbl`

17. 打开文件: `vim arch/arm64/tools/syscall_64.tbl` ;

18. 同样取“最后一行调用号+1”作为新调用号 (示例 470) ;

19. 在末尾添加:

代码块

```
1 470 common schello sys_schello
```

20. 保存退出: `:wq` 。

Step4: 更新系统调用号定义 (通用配置)

目的: 在用户态可见的头文件中定义新调用号, 确保用户程序能通过 `syscall(调用号)` 触发调用。

操作文件: `include/uapi/asm-generic/unistd.h`

具体步骤:

21. 打开文件: `vim include/uapi/asm-generic/unistd.h` ;

22. 定位到**第895行附近** (`__NR_syscalls` 定义附近) ;

23. 按以下方式修改 (注意调用号与Step3保持一致, 示例为 470) :

代码块

```
1 // 1. 定义新系统调用号__NR_schello
2 #define __NR_schello 470
3 // 2. 将调用号与函数关联
4 __SYSCALL(__NR_schello, sys_schello)
5
6 // 3. 更新系统调用总数 (原__NR_syscalls+1, 示例从470改为471)
7 #undef __NR_syscalls
8 #define __NR_syscalls 471
```

24. 保存退出: `:wq` 。

Step5: 编译并安装新内核

目的: 将修改后的内核源码编译为可启动的内核镜像, 并安装到系统中。

操作命令（在 kernel 源码根目录执行，需root权限）：

25. 清理旧编译产物（避免冲突）：

代码块

```
1 sudo make clean
```

26. 多线程编译内核（`-j$(nproc)` 表示用所有CPU核心加速，`2>&1 | tee log` 将输出保存到log文件）：

代码块

```
1 sudo make -j$(nproc) 2>&1 | tee kernel_build.log
```

注意：编译耗时较长（约30分钟~2小时，取决于硬件性能），需确保磁盘空间充足（至少20GB）。

27. 安装内核模块：

代码块

```
1 sudo make modules_install
```

28. 安装新内核（更新 grub 启动项）：

代码块

```
1 sudo make install
```

Step6：重启系统并验证内核版本

目的：让系统加载新编译的内核，并确认内核版本正确。

29. 重启系统：

代码块

```
1 sudo reboot
```

30. 重启后执行以下命令，查看内核版本是否为 `6.17.1`：

代码块

```
1  uname -a
```

代码块

```
1  Linux ubuntu 6.17.1 #1 SMP PREEMPT Wed Oct 29 15:30:00 CST 2024 x86_64
   x86_64 x86_64 GNU/Linux
```

若版本不符，需检查Step5编译是否成功，或grub启动项是否选择了新内核。

Step7: 编写用户态测试程序

目的：编写用户程序，通过 `syscall(调用号)` 触发 `sys_schello` 系统调用。

操作步骤：

31. 在任意目录（如 `~/test`）创建文件 `testschello.c`：

代码块

```
1  mkdir -p ~/test && cd ~/test
2  vim testschello.c
```

32. 添加以下代码（调用号需与Step3/Step4的 `470` 一致）：

代码块

```
1  #include <unistd.h>    // 包含syscall()函数声明
2  #include <sys/syscall.h> // 系统调用相关定义
3  #include <stdio.h>      // 包含printf()
4
5  int main(int argc, char *argv[]) {
6      // 调用新系统调用：参数为Step3定义的调用号470
7      syscall(470);
8      // 提示用户查看内核日志
9      printf("ok! Run the cmd to check kernel log: sudo dmesg | grep
   schello\n");
10     return 0;
11 }
```

33. 保存退出：`:wq`。

Step8: 编译并运行测试程序

目的：执行用户程序，验证新系统调用是否生效。

操作命令（在 `~/test` 目录执行）：

34. 编译测试程序（生成可执行文件 `testsc`）：

代码块

```
1 gcc -o testsc testschello.c
```

35. 运行测试程序（无需root权限，因 `syscall()` 本身允许用户态调用）：

代码块

```
1 ./testsc
```

代码块

```
1 ok! Run the cmd to check kernel log: sudo dmesg | grep schello
```

36. 查看内核日志（`dmesg` 用于查看内核态打印，`grep schello` 过滤目标日志）：

代码块

```
1 sudo dmesg | grep schello
```

代码块

```
1 [1234.567890] Hello new system call schello! Linux Kernel Version: 6.17.1
2 [1234.567901] Hello new system call schello! Student ID: 123456 # 实际学号
```

四、实验结果

37. 内核版本验证： `uname -a` 显示系统运行 `linux-6.17.1` 内核，证明新内核安装成功；

38. 系统调用触发：运行 `./testsc` 后， `sudo dmesg | grep schello` 能正确输出包含“Linux Kernel Version: 6.17.1”和个人学号的日志，证明 `sys_schello` 系统调用在 kernel 态正常执行；

39. 用户态交互：测试程序无报错，且正确提示查看日志的命令，证明用户态与内核态的系统调用交互正常。

```

gww@gww@gww-VMware-Virtual-Platform:~/os/第五次$ gcc -o testsc testschello.c
Linuxgww@gww-VMware-Virtual-Platform:~/os/第五次$ ./testsc
2 16:ok! Run the cmd to check kernel log: sudo dmesg | grep schello
gww@gww@gww-VMware-Virtual-Platform:~/os/第五次$ sudo dmesg | grep schello
[sudo] gww 的密码:
[ 133.993024] Hello new system call schello! Your ID
[ 133.993029] Hello new system call schello! Hello 2414308
gww@gww-VMware-Virtual-Platform:~/os/第五次$ uname -a
Linux gww-VMware-Virtual-Platform 6.17.12414308 #3 SMP PREEMPT_DYNAMIC Sun Nov
2 16:44:10 CST 2025 x86_64 x86_64 x86_64 GNU/Linux
gww@gww-VMware-Virtual-Platform:~/os/第五次$ █

```

```

/*
 * Not a real system call, but a placeholder for syscalls which are
 * not implemented -- see kernel/sys_ni.c
 */
asmlinkage long sys_ni_syscall(void);
asmlinkage long sys_schello(void);
#endif /* CONFIG_ARCH_HAS_SYSCALL_WRAPPER */

asmlinkage long sys_ni_posix_timers(void);

/*
 * Kernel code should not call syscalls (i.e., sys_xyzzyz()) directly.
 * Instead, use one of the functions which work equivalently, such as
 * the ksys_xyzzyz() functions prototyped below.
-- 插入 --

```

1219,35

92%

```

{
    return task_pid_vnr(current);
}
SYSCALL_DEFINE0(schello)
{
    printk(KERN_INFO "Hello new system call schello! 2414308\n");
    printk(KERN_INFO "Hello new system call schello! Hello 123456\n");
    return 0;
}

/*
 * Accessing ->real_parent is not SMP-safe, it could
 * change from under us. However, we can use a stale
 * value of ->real_parent under rcu_read_lock(), see
 * release_task()->call_rcu(delayed_put_task_struct).
 */

```

1013,1-8

33%


```
457 common statmount sys_statmount
458 common listmount sys_listmount
459 common lsm_get_self_attr sys_lsm_get_self_attr
460 common lsm_set_self_attr sys_lsm_set_self_attr
461 common lsm_list_modules sys_lsm_list_modules
462 common mseal sys_mseal
463 common setxattrat sys_setxattrat
464 common getxattrat sys_getxattrat
465 common listxattrat sys_listxattrat
466 common removexattrat sys_removexattrat
467 common open_tree_attr sys_open_tree_attr
468 common file_getattr sys_file_getattr
469 common file_setattr sys_file_setattr
470 common schello sys_schello
-- 插入 --
```

413,8 底端

五、实验总结

1. 实验成功关键

- **调用号一致性：**Step3（系统调用表）、Step4（调用号定义）、Step7（用户程序 `syscall()`）的调用号必须完全一致（如示例 470），否则内核无法匹配函数；
- **内核编译完整性：**`make modules_install` 和 `make install` 必须执行，否则新内核无法被grub识别；
- **日志级别与权限：**`printk` 使用 `KERN_INFO` 级别（确保日志不被过滤），`dmesg` 需 `sudo` 权限（内核日志仅root可完全查看）。

2. 常见问题与解决方案

问题现象	可能原因	解决方案
编译内核时提示“头文件缺失”	<code>build-essential</code> 未安装或版本不足	重新执行 <code>sudo apt-get install build-essential</code>
重启后 <code>uname -a</code> 仍显示旧内核	grub未默认选择新内核	重启时按 <code>Shift</code> 进入grub菜单，手动选择新内核
<code>`dmesg</code>	<code>grep schello`</code> 无输出	调用号不匹配或 <code>printk</code> 日志级别错误
运行 <code>./testsc</code> 提示“权限不足”	测试程序无执行权限	执行 <code>chmod +x testsc</code> 添加执行权限
虚拟机连不上网络		

	1. 重启虚拟机的 NAT 服务 (主机侧) 在你的物理机 (原操作系统) 上:	1. 重启虚拟机的 NAT 服务 (主机侧) 在你的物理机 (原操作系统) 上: - 打开 “服 务” (Windows 按 Win+R 输入 services.msc) ; - 找到 VMware NAT Service , 右键 “重启” ; - 重启后再回到虚拟 机, 尝试重新联网。
--	---	--

3. 实验意义

通过手动添加系统调用，深入理解了Linux “用户态-内核态” 的隔离与交互机制，掌握了内核源码修改、编译、安装的完整流程，为后续内核开发（如驱动开发、系统优化）奠定基础。